```
using Logging
using Random
using CompetingClocks:
    SSA, CombinedNextReaction, enable!, disable!, next, keytype, steploglikelihood

using Distributions

export SimulationFSM

########## The Simulation Finite State Machine (FSM)

mutable struct SimulationFSM{State,Sampler,CK}
    physical::State
    sampler::Sampler
    eventgen::GeneratorSearch
    immediategen::GeneratorSearch
    when::Float64
    rng::Xoshiro
    depnet::DependencyNetwork{CK}
    enabled_events::Dict{CK,SimEvent}
    enabling_times::Dict{CK,Float64}
    observer
end


"""
Look at events and determine a common base type.
Internally the simulation tracks events with sets of tuples by turning
each event instance into a tuple. If all the tuples have the same type,
this should turn out to be performant.
"""
function common_base_key_tuple(events)
    all_field_types = [Tuple{Symbol,fieldtypes(T)...} for T in events]
    typejoined = reduce(typejoin, all_field_types)
    return typejoined
end


"""
    SimulationFSM(physical_state, trans_rules; seed, rng, sampler, observer=nothing
)

Create a simulation.

The `physical_state` is of type `PhysicalState`. The sampler is of type
`CompetingClocks.SSA`. The `trans_rules` are a list of type `SimEvent`.
The seed is an integer seed for a `Xoshiro` random number generator. The
observer is a callback with the signature:

```
observer(physical, when::Float64, event::SimEvent, changed_places::AbstractSet{Tupl
e})
```

The `changed_places` argument is a set-like object with tuples that are keys that
represent which places were changed.
"""
function SimulationFSM(
    physical, events; sampler=nothing, observer=nothing, rng=nothing, seed=nothing
)
    randgen = if !isnothing(rng)
        rng
    elseif !isnothing(seed)
        Xoshiro(seed)
    else
        Xoshiro()
    end
    if isnothing(sampler)
        ClockKey = common_base_key_tuple(events)
        sampler = CombinedNextReaction{ClockKey,Float64}()
        @debug "Creating a sampler with clock key type $ClockKey"
    else
        ClockKey = keytype(sampler)
    end
    no_generator_event = Any[]
    generator_searches = Dict{String,GeneratorSearch}()
    for (idx, filter_condition) in Dict("timed" => !isimmediate, "immediate" => isi
mmediate)
        event_set = filter(filter_condition, events)
        generator_set = EventGenerator[]
        for event in event_set
            gen_for_event = generators(event)
            if !isempty(gen_for_event)
                append!(generator_set, gen_for_event)
            else
                push!(no_generator_event, gen_for_event)
            end
        end
        generator_searches[idx] = GeneratorSearch(generator_set)
    end
    if isempty(generator_searches["timed"])
        imm_str = str(generator_searches["immediate"])
        error("There are no timed events and immediate events are $imm_str")
    end
    if length(no_generator_event) > 1
        error("""More than one event has no generators. Check function signatures
            because only one should be the initializer event. $(no_generator_event)
            """)
    elseif !isempty(no_generator_event)
        @debug "Possible initialization event $(no_generator_event[1])"
    end
    @debug generator_searches["timed"]

    if isnothing(observer)
        observer = (args...) -> nothing
    end
    return SimulationFSM{typeof(physical),typeof(sampler),ClockKey}(
        physical,
        sampler,
        generator_searches["timed"],
        generator_searches["immediate"],
        0.0,
        randgen,
        DependencyNetwork{ClockKey}(),
        Dict{ClockKey,SimEvent}(),
        Dict{ClockKey,Float64}(),
        observer,
)
end

checksim(sim::SimulationFSM) = @assert keys(sim.enabled_events) == keys(sim.depnet.
event)

function rate_reenable(sim::SimulationFSM, event, clock_key)
    first_enable = sim.enabling_times[clock_key]
    reads_result = capture_state_reads(sim.physical) do
        return reenable(event, sim.physical, first_enable, sim.when)
    end
    if !isnothing(reads_result.result)
        (dist, enable_time) = reads_result.result
        enable!(sim.sampler, clock_key, dist, enable_time, sim.when, sim.rng)
```

```
    end
    return reads_result.reads
end


function process_generated_events_from_changes(sim::SimulationFSM, fired_event_key,
 changed_places)
    over_generated_events(sim.eventgen, sim.physical, fired_event_key, changed_plac
es) do newevent
        evtkey = clock_key(newevent)
        if evtkey â\210\211 keys(sim.enabled_events)
            precond = capture_state_reads(sim.physical) do
                result = precondition(newevent, sim.physical)
                if isnothing(result)
                    error("""The precondition for $newevent returned `nothing` whic
h may
                        mean that the precondition function doesn't return a true/f
alse or
                        that the interface stub for precondition was called because
 the
                        function signature for $(newevent)'s precondition doesn't m
atch.
                        """)
                end
                return result
            end
            if precond.result
                input_places = precond.reads
                sim.enabled_events[evtkey] = newevent
                sim.enabling_times[evtkey] = sim.when
                reads_result = capture_state_reads(sim.physical) do
                    enabling_spec = enable(newevent, sim.physical, sim.when)
                    if length(enabling_spec) != 2
                        error("""The enable() function for $newevent should return
a
                            distribution and a time. This one returns $enabling_spe
c.
                            """)
                    end
                    (dist, enable_time) = enabling_spec
                    enable!(sim.sampler, evtkey, dist, enable_time, sim.when, sim.r
ng)
                end
                rate_deps = reads_result.reads
                @debug "Evtkey $(evtkey) with enable deps $(input_places) rate deps
 $(rate_deps)"
                add_event!(sim.depnet, evtkey, input_places, rate_deps)
            end
        end
    end
end


"""
    deal_with_changes(sim::SimulationFSM)

An event changed the state. This function modifies events
to respond to changes in state.
"""
function deal_with_changes(
    sim::SimulationFSM{State,Sampler,CK}, fired_event, changed_places
) where {State,Sampler,CK}
    # This function starts with enabled events. It ends with enabled events.
    # Let's look at just those events that depend on changed places.
    #                      Finish
    #                 Enabled     Disabled
    # Start  Enabled  re-enable   remove
    #        Disabled  create      nothing
    #
    # Sort for reproducibility run-to-run.
    @debug "Fired $(fired_event) changed $(changed_places)"
    if !isempty(changed_places)
        clock_toremove = CK[]
        cond_affected = union((getplace_enable(sim.depnet, cp) for cp in changed_pl
aces)...)
        rate_affected = union((getplace_rate(sim.depnet, cp) for cp in changed_plac
es)...)

        for check_clock_key in sort(collect(cond_affected))
            event = sim.enabled_events[check_clock_key]
            reads_result = capture_state_reads(sim.physical) do
                precondition(event, sim.physical)
            end
            cond_result = reads_result.result
            cond_places = reads_result.reads

            if !cond_result
                push!(clock_toremove, check_clock_key)
            else
                # Every time we check an invariant after a state change, we must
                # re-calculate how it depends on the state. For instance,
                # A can move right. Then A moves down. Then A can still move
                # right, but its moving right now depends on a different space
                # to the right. This is because a "move right" event is defined
                # relative to a state, not on a specific set of places.
                if cond_places != getplace_enable(sim.depnet, check_clock_key)
                    # Then you get new places.
                    rate_deps = rate_reenable(sim, event, check_clock_key)
                    add_event!(sim.depnet, check_clock_key, cond_places, rate_deps)
                    if check_clock_key in rate_affected
                        delete!(rate_affected, check_clock_key)
                    end
                end
            end
        end

        for rate_clock_key in sort(collect(rate_affected))
            event = sim.enabled_events[rate_clock_key]
            rate_deps = rate_reenable(sim, event, rate_clock_key)
            cond_deps = getplace_enable(sim.depnet, rate_clock_key)
            add_event!(sim.depnet, rate_clock_key, cond_deps, rate_deps)
        end

        # Split the loop over changed_places so that the first part disables clocks
        # and the second part creates new ones. We do this because two clocks
        # can have the SAME key but DIFFERENT dependencies. For instance, "move lef
t"
        # will depend on different board places after the piece has moved.
        disable_clocks!(sim, clock_toremove)
    end
end


function disable_clocks!(sim::SimulationFSM, clock_keys)
    isempty(clock_keys) && return nothing
    @debug "Disable clock $(clock_keys)"
    for clock_done in clock_keys
        disable!(sim.sampler, clock_done, sim.when)
        delete!(sim.enabled_events, clock_done)
        delete!(sim.enabling_times, clock_done)
    end
    remove_event!(sim.depnet, clock_keys)
end
```

```
function modify_state!(sim::SimulationFSM, fire_event)
    changes_result = capture_state_changes(sim.physical) do
        fire!(fire_event, sim.physical, sim.when, sim.rng)
    end
    changed_places = changes_result.changes
    seen_immediate = SimEvent[]
    over_generated_events(
        sim.immediategen, sim.physical, clock_key(fire_event), changed_places
    ) do newevent
        if newevent â\210\211 seen_immediate && precondition(newevent, sim.physical
)
            push!(seen_immediate, newevent)
            ans = capture_state_changes(sim.physical) do
                fire!(newevent, sim.physical, sim.when, sim.rng)
            end
            push!(changed_places, ans.changes)
        end
    end
    return changed_places
end

"""
    fire!(sim::SimulationFSM, time, event_key)

Let the event act on the state.
"""
function fire!(sim::SimulationFSM, when, what)
    sim.when = when
    event = sim.enabled_events[what]
    # Break the invariant that state and events are consistent.
    changed_places = modify_state!(sim, event)
    disable_clocks!(sim, [what])
    deal_with_changes(sim, event, changed_places)
    process_generated_events_from_changes(sim, what, changed_places)
    checksim(sim)
    # Invariant for states and events is restored, so show the result.
    sim.observer(sim.physical, when, event, changed_places)
end

get_enabled_events(sim::SimulationFSM) = collect(values(sim.enabled_events))

"""
Initialize the simulation. You could call it as a do-function.
It is structured this way so that the simulation will record changes to the
physical state.
```
    initialize!(sim) do init_physical
        initialize!(init_physical, agent_cnt, sim.rng)
    end
```
"""
function initialize!(init_evt, callback::Function, sim::SimulationFSM)
    changes_result = capture_state_changes(sim.physical) do
        callback(sim.physical, sim.when, sim.rng)
    end
    deal_with_changes(sim, init_evt, changes_result.changes)
    process_generated_events_from_changes(sim, clock_key(init_evt), changes_result.
changes)
    checksim(sim)
    sim.observer(sim.physical, sim.when, init_evt, changes_result.changes)
end


"""
    run(simulation, initializer, stop_condition)

Given a simulation, this initializes the physical state and generates a
trajectory from the simulation until the stop condition is met. The `initializer`
is either a function whose argument is a physical state and returns nothing, or
it is an event key for an event that initializes the system. The
stop condition is a function with the signature:

```
stop_condition(physical_state, step_idx, event::SimEvent, when)::Bool
```

The event and when passed into the stop condition are the event and time that are
about to fire but have not yet fired. This lets you enforce a stopping time that
is between events.
"""
function run(sim::SimulationFSM, init_evt::SimEvent, init_func::Function, stop_cond
ition::Function)
    step_idx = 0
    initialize!(init_evt, init_func, sim)
    stop_condition(sim.physical, step_idx, init_evt, sim.when) && return nothing
    step_idx += 1
    while true
        (when, what) = next(sim.sampler, sim.when, sim.rng)
        if isfinite(when) && !isnothing(what)
            stop_condition(sim.physical, step_idx, what, when) && break
            @debug "Firing $what at $when"
            fire!(sim, when, what)
        else
            @info "No more events to process after $step_idx iterations."
            break
        end
        step_idx += 1
    end
    step_idx
end

function run(sim::SimulationFSM, init_evt::SimEvent, stop_condition::Function)
    init_func = (physical, when, rng) -> fire!(init_evt, physical, when, rng)
    run(sim, init_evt, init_func, stop_condition)
end

function run(sim::SimulationFSM, initializer::Function, stop_condition::Function)
    run(sim, InitializeEvent(), initializer, stop_condition)
end

"""
The `trace` is a `Vector{Tuple{Float64,SimEvent}}`. That is, it's a list of
tuples containing `(when, what event)`.
In order to calculate log-likelihood of a simulation, pass it a sampler that
tracks log-likelihood. For instance,
```julia
base_sampler = CombinedNextReaction{K,T}()
memory_sampler = MemorySampler(base_sampler)
```
"""
function trace_likelihood(sim::SimulationFSM, init_evt::SimEvent, init_func::Functi
on, trace)
    loglikelihood = zero(Float64)
    initialize!(init_evt, init_func, sim)
    for (step_idx, step_evt) in enumerate(trace)
        (when, what) = step_evt
        if isfinite(when) && !isnothing(what)
            @debug "Firing $what at $when"
            @assert when > sim.when
            loglikelihood += steploglikelihood(sim.sampler.track, sim.when, when, w
hat)
            fire!(sim, when, what)
        else
```

```
            @info "No more events to process after $step_idx iterations."
            break
        end
    end
    loglikelihood
end

function trace_likelihood(sim::SimulationFSM, initializer::SimEvent, trace)
    init_func = (physical, when, rng) -> fire!(initializer, physical, when, rng)
    trace_likelihood(sim, initializer, init_func, trace)
end

function trace_likelihood(sim::SimulationFSM, initializer::Function, trace)
    trace_likelihood(sim, InitializeEvent(), initializer, trace)
end
```